# Evidencias del proyecto final

## Pronóstico de series de tiempo en producción

| | |
|---|---|
| Curso | Computación Avanzada (ET0197), Institución Universitaria Pascual Bravo |
| Estudiante | Nicolas Cataño Durango |
| Repositorio | github.com/TekilaNCD/Proyecto_CA |
| Fecha | 8 de noviembre de 2026 |

Este documento recoge, con capturas de pantalla, cómo se construyó el servicio: un modelo SARIMA entrenado en Google Colab con datos de InfluxDB, una API en FastAPI con un frontend propio, y su empaquetado en un contenedor Docker. **Las credenciales de InfluxDB (token, URL, organización y bucket) están ocultas con barras negras en todas las capturas.**

### Resumen de resultados

| Elemento | Resultado |
|---|---|
| Serie | Measurement **Server**, field **Consumo_total**. 720 horas (30 días), 0 horas rellenadas |
| Modelo | SARIMA(2,0,2)x(1,1,1,24), elegido por AIC (7265.3) entre 36 combinaciones |
| Evaluación (últimas 24 h) | MAE 68.4 vs 83.1 de la línea base ingenua estacional. RMSE 102.2 vs 119.3. MAPE 21.9 % vs 27.8 % |
| Residuos | Ljung-Box con p = 0.77 (rezago 24) y p = 0.97 (rezago 48): sin autocorrelación |
| Verificación del modelo | Diferencia máxima API vs. modelo original: 0.0 |
| App sin Docker | "modelo_demo": false, atendido_por = codespaces-b9580d |
| App con Docker | atendido_por = CONTAINER ID = 4fc45f823e5a; la imagen no contiene credenciales |

# 1. Preparación del entorno (Fase 0)

Se creó el repositorio desde la plantilla y se abrió un Codespace sobre la rama main. Las credenciales de InfluxDB se guardaron como secrets del Codespace, nunca dentro del código.

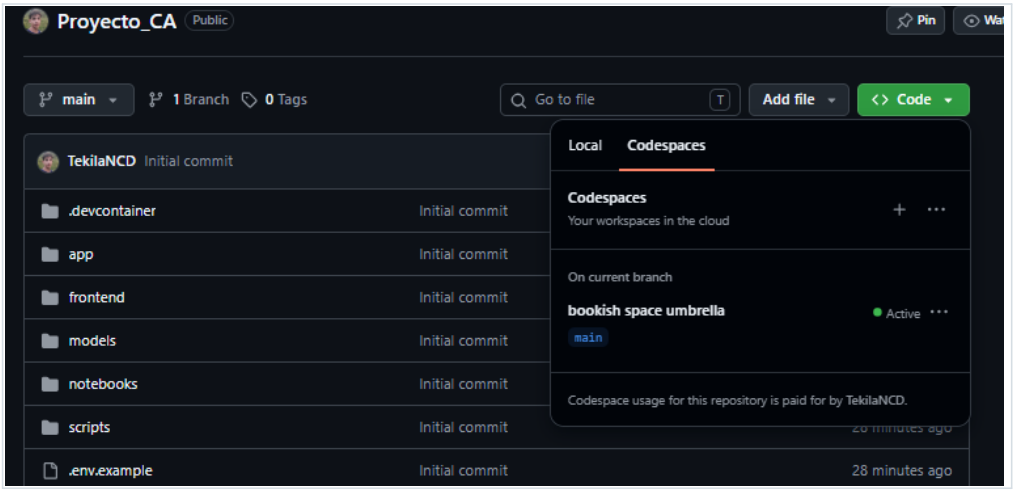



*Figura 1. Repositorio Proyecto_CA con el Codespace activo.*

Con el script de prueba se confirmó la conexión y se identificó la serie a pronosticar.

```
$ INFLUX_MEASUREMENT=Server INFLUX_FIELD=Consumo_total python scripts/probar_conexion.py
Ping: OK

Measurements en '[bucket oculto]' (últimos 30 días): ['Server']
Fields de 'Server': ['Consumo_total']

Serie horaria (Server/Consumo_total): 72 horas
tiempo
2026-11-08 12:00:00     178.901854
2026-11-08 13:00:00     187.711188
2026-11-08 14:00:00     191.695379
2026-11-08 15:00:00     206.978780
2026-11-08 16:00:00     375.741614
2026-11-08 17:00:00     640.431168
2026-11-08 18:00:00    1015.332235
2026-11-08 19:00:00     867.110307
2026-11-08 20:00:00     375.634232
2026-11-08 21:00:00     247.795791
Freq: h, Name: valor, dtype: float64
```

*Salida de la terminal del Codespace: conexión correcta y serie con datos.*

# 2. Modelo SARIMA en Google Colab (Fase 1)

## Configuración y datos

Los secrets se crearon en Colab con el acceso al cuaderno activado. Los valores están ocultos.

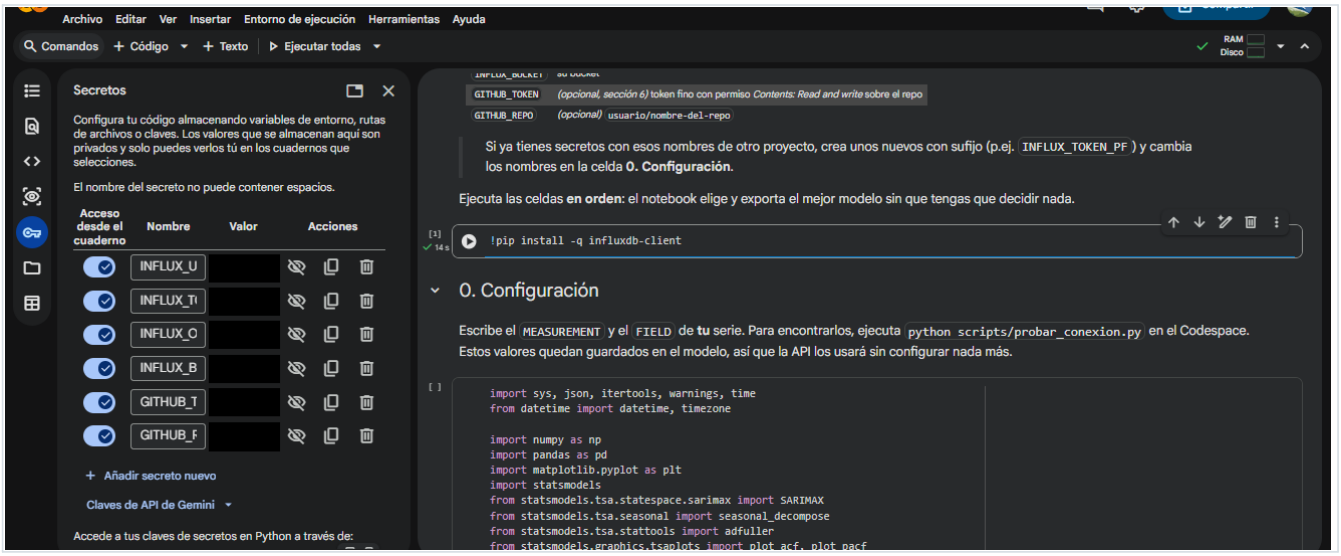



*Figura 2. Secrets de Colab con acceso al cuaderno activado (valores ocultos).*

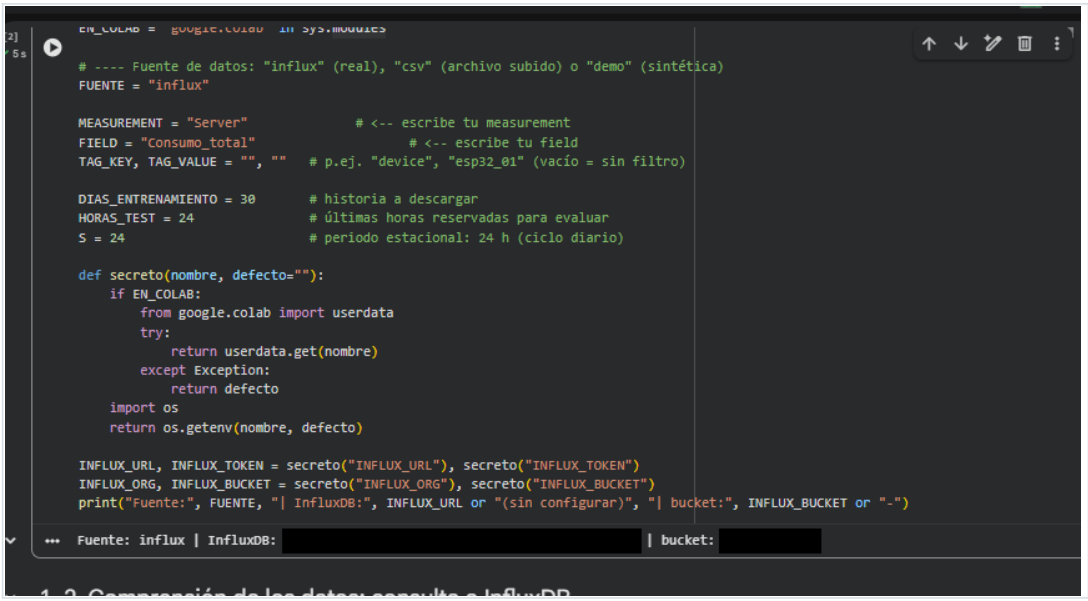



*Figura 3. Celda de configuración: MEASUREMENT = "Server" y FIELD = "Consumo_total" (URL y bucket ocultos).*

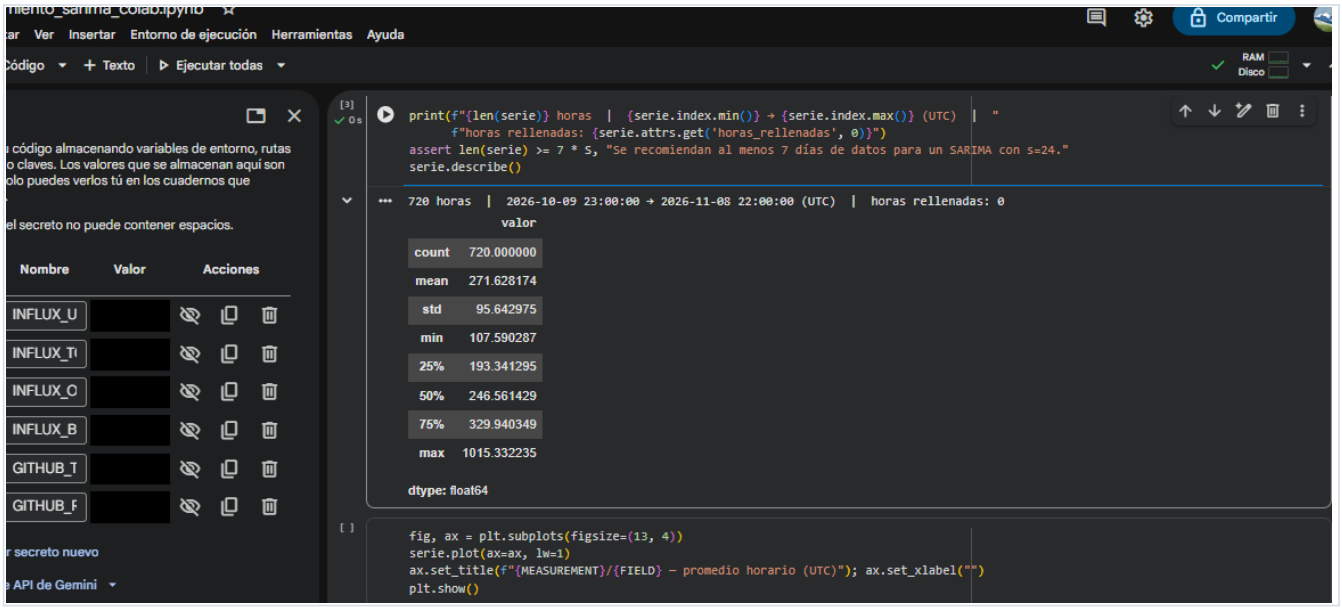



*Figura 4. La consulta a InfluxDB trae 720 horas, del 9 de octubre al 8 de noviembre, sin horas rellenadas.*

## Comprensión de los datos




*Figura 5. Serie horaria de Server/Consumo_total.*

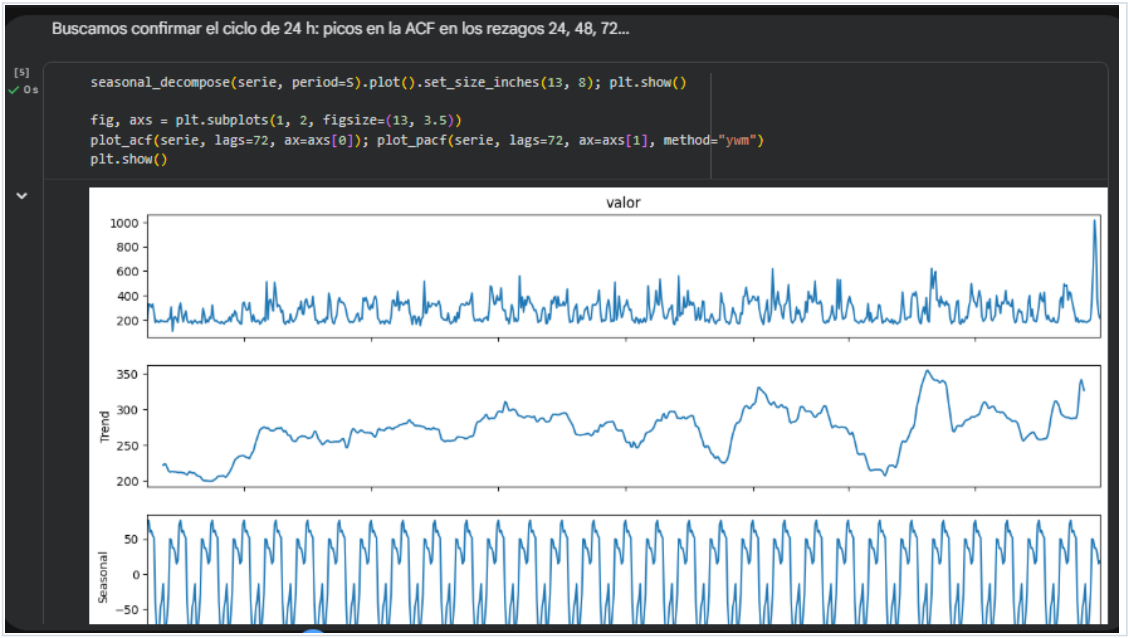



*Figura 6. Descomposición con periodo de 24 horas: se ve claramente el componente estacional.*

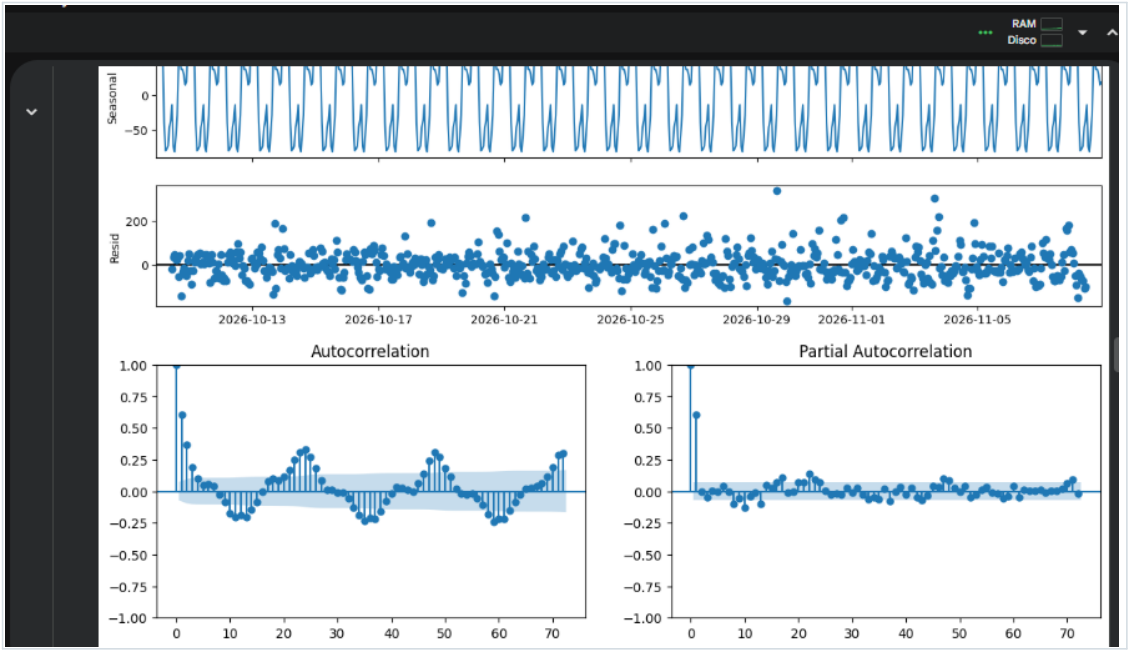



*Figura 7. Residuo de la descomposición, ACF y PACF. La ACF tiene picos en los rezagos 24, 48 y 72, lo que confirma el ciclo diario (S = 24).*

## Preparación

Con las cercas de Tukey (K = 3) se detectaron 2 atípicos extremos, el 8 de noviembre a las 18:00 y 19:00 UTC (1015 y 867), que se reemplazaron por interpolación. La prueba ADF dio p = 0.0000, por lo que d = 0.

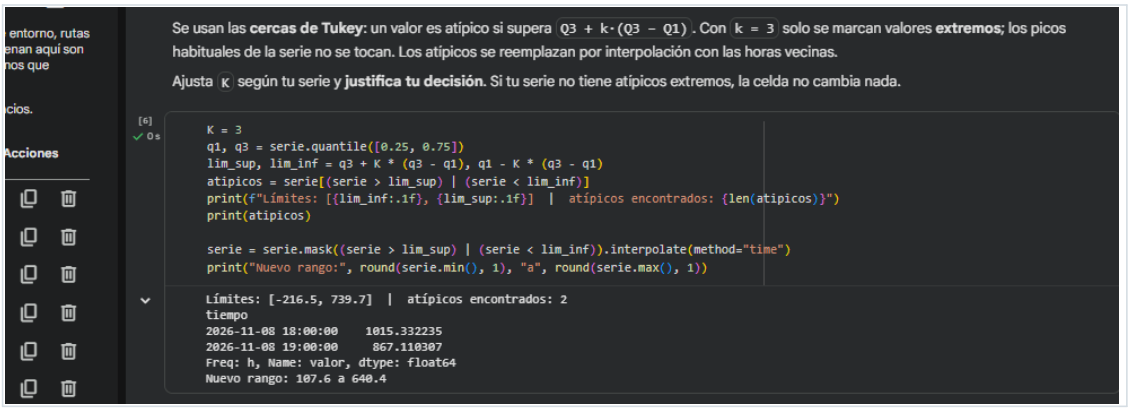



*Figura 8. Detección y tratamiento de atípicos.*

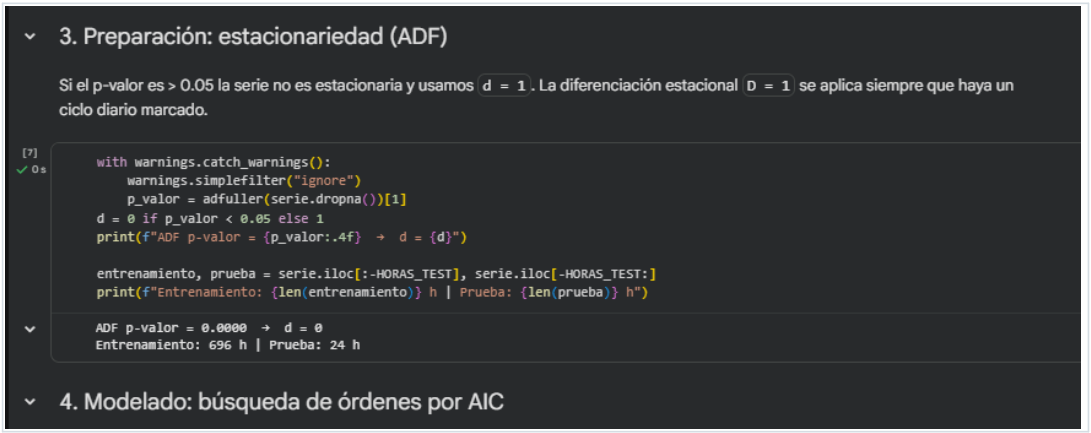



*Figura 9. Prueba ADF y división en 696 horas de entrenamiento y 24 de prueba.*

## Modelado y evaluación

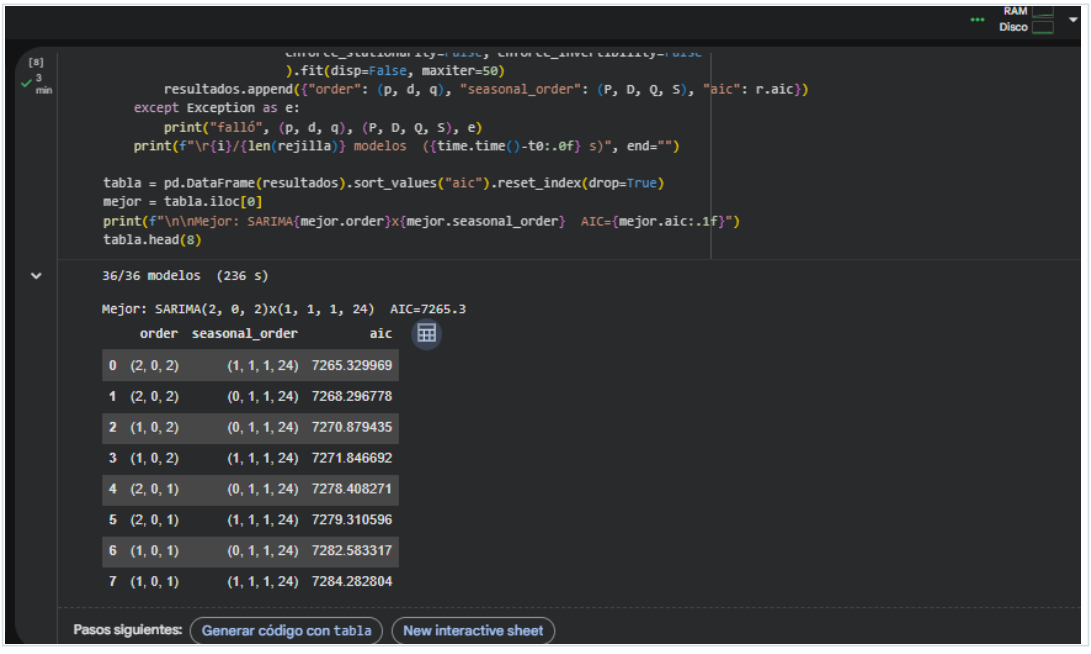



*Figura 10. Búsqueda por AIC de 36 modelos. Mejor: SARIMA(2,0,2)x(1,1,1,24), AIC = 7265.3.*

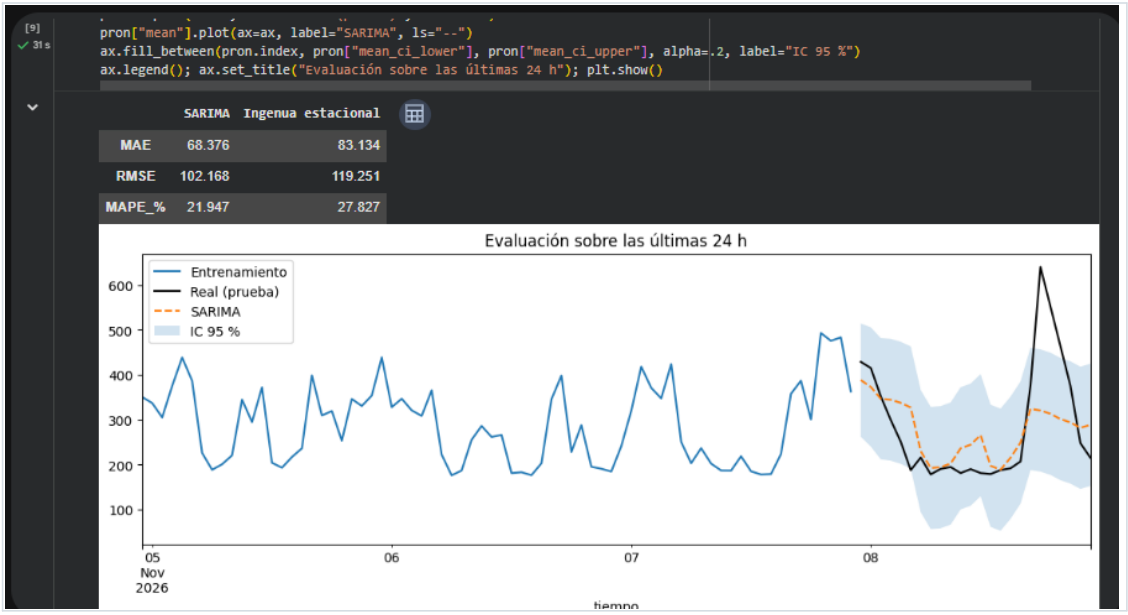



*Figura 11. Evaluación sobre las últimas 24 horas: el SARIMA supera a la línea base ingenua estacional en MAE, RMSE y MAPE.*

## Reentrenamiento y diagnóstico de residuos

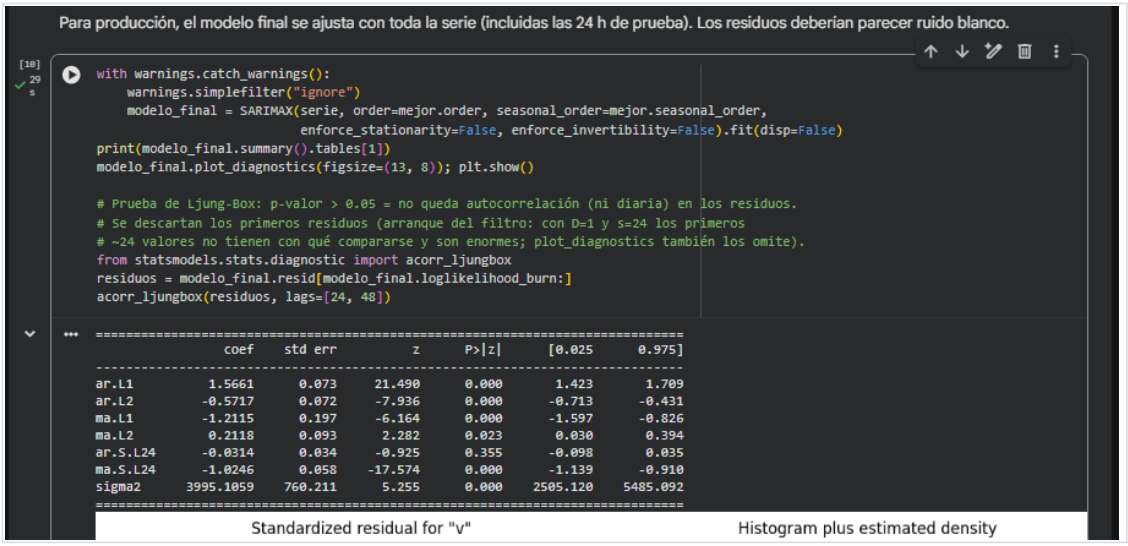



*Figura 12. Modelo final ajustado con toda la serie: coeficientes estimados.*

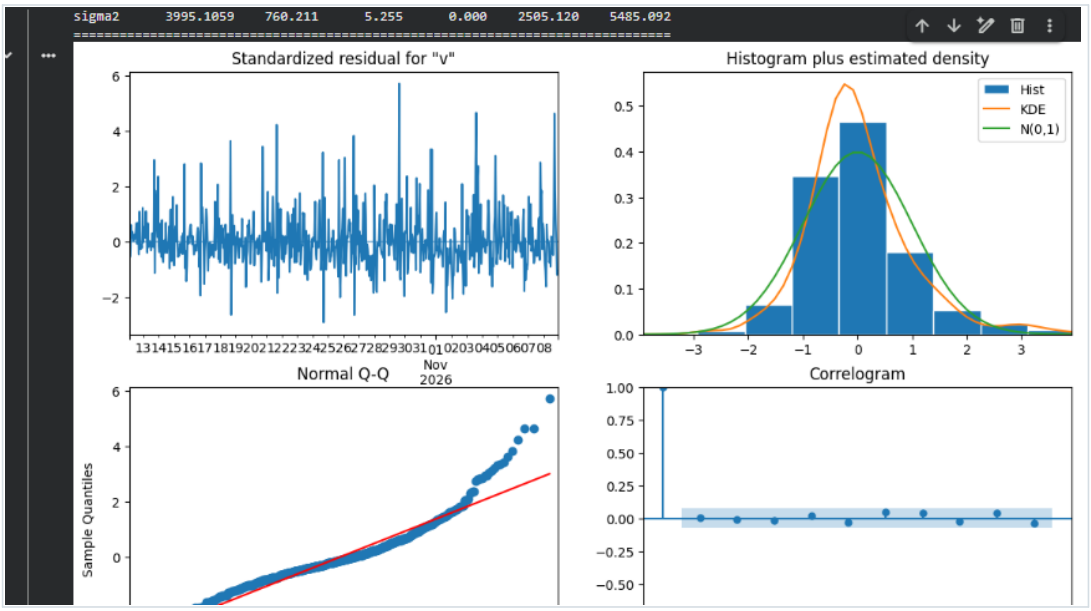



*Figura 13. Diagnóstico de residuos: residuos estandarizados, histograma y correlograma.*

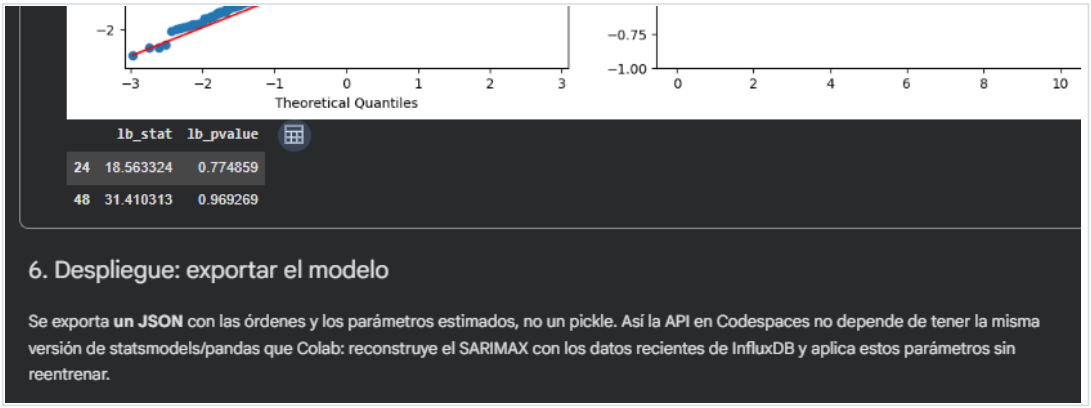



*Figura 14. Gráfico Q-Q y prueba de Ljung-Box (p = 0.77 y 0.97, ambos mayores a 0.05).*

## Exportación y verificación

El modelo se exportó como JSON (órdenes y parámetros, sin pickle). La verificación reconstruye el SARIMAX con esos parámetros y compara el pronóstico contra el modelo original.

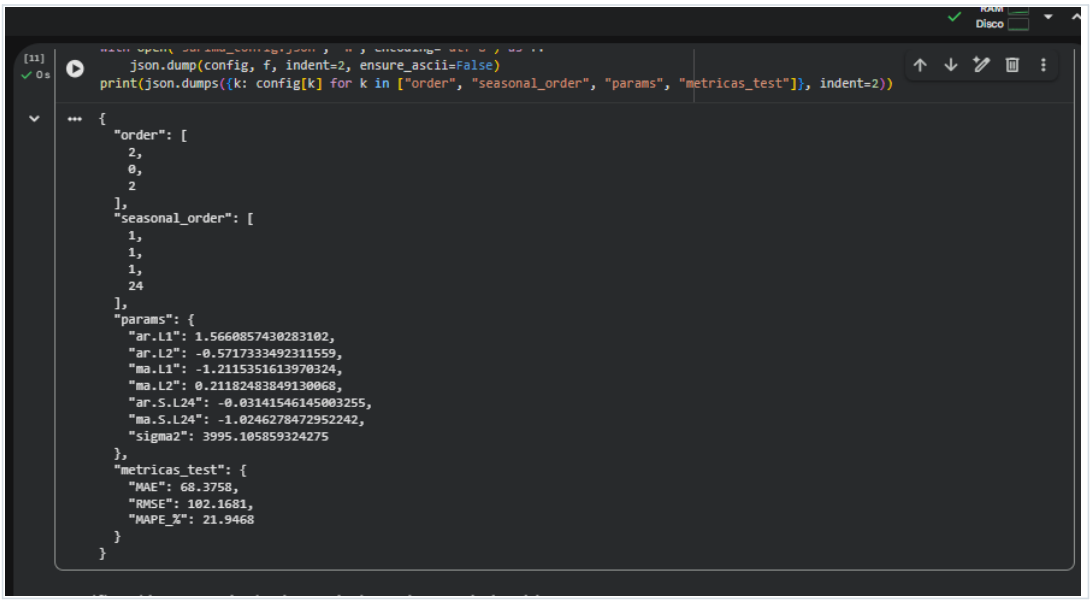



*Figura 15. Contenido exportado de sarima_config.json.*

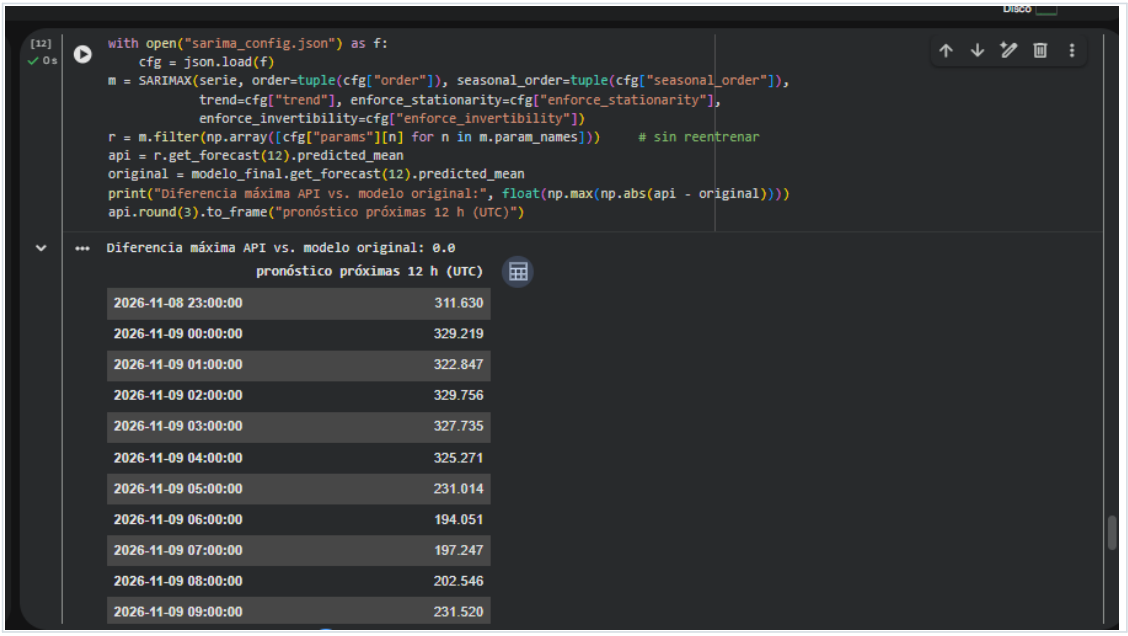



*Figura 16. Verificación: "Diferencia máxima API vs. modelo original: 0.0".*

## Subida al repositorio

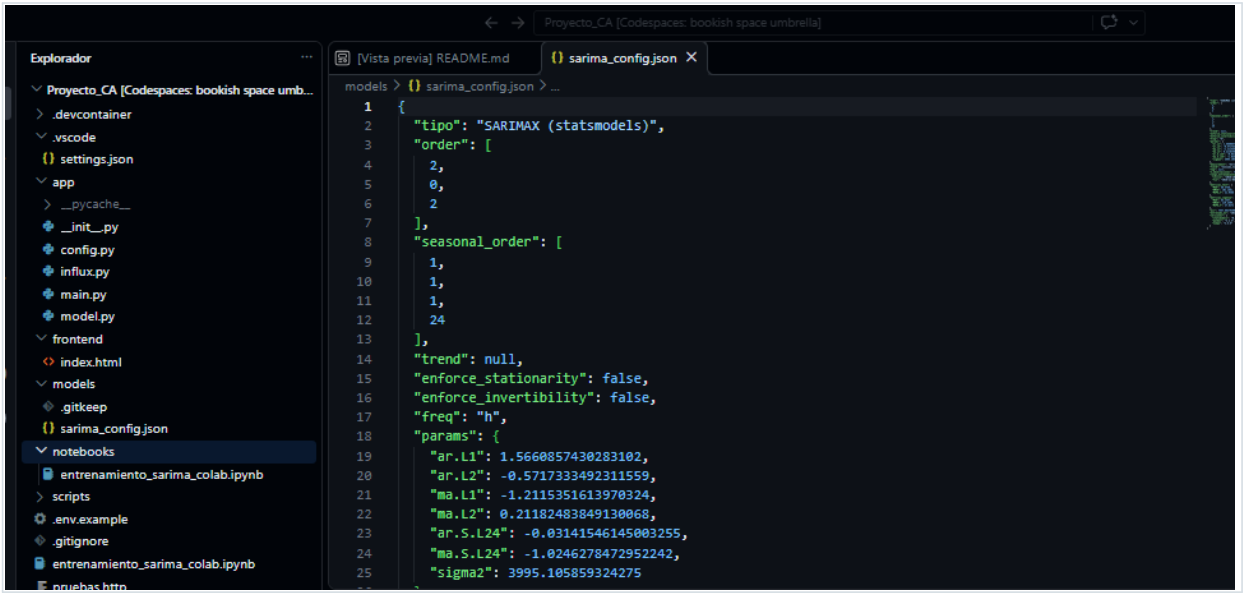



*Figura 17. models/sarima_config.json dentro del repositorio, en el Codespace.*

```
$ git add models/sarima_config.json
$ git commit -m "Modelo SARIMA entrenado en Colab"
[main 7943e0d] Modelo SARIMA entrenado en Colab
 1 file changed, 52 insertions(+)
 create mode 100644 models/sarima_config.json
$ git push
   f33c0b5..7943e0d  main -> main

$ mv -f entrenamiento_sarima_colab.ipynb notebooks/entrenamiento_sarima_colab.ipynb
$ git add -A
$ git commit -m "Notebook ejecutado en notebooks/"
[main bdb2420] Notebook ejecutado en notebooks/
$ git push
   1738d12..bdb2420  main -> main
```

*Modelo y notebook ejecutado subidos a GitHub (salida abreviada).*

# 3. Frontend y app sin Docker (Fases 2 y 3)

El frontend reemplaza el esqueleto de `frontend/index.html` y cumple los cinco requisitos: gráfica con histórico, pronóstico e intervalo de confianza; horas en hora de Colombia (UTC-5); un indicador destacado; control de las horas a pronosticar; y un mensaje claro cuando la API responde con error.




*Figura 18. Indicador destacado (valor de la próxima hora con su intervalo), máximo esperado, último dato recibido y calidad del dato. Horas en hora de Colombia.*




*Figura 19. Gráfica con histórico, pronóstico (punteado) e intervalo de confianza del 95 %, con los controles de horas, histórico y confianza.*

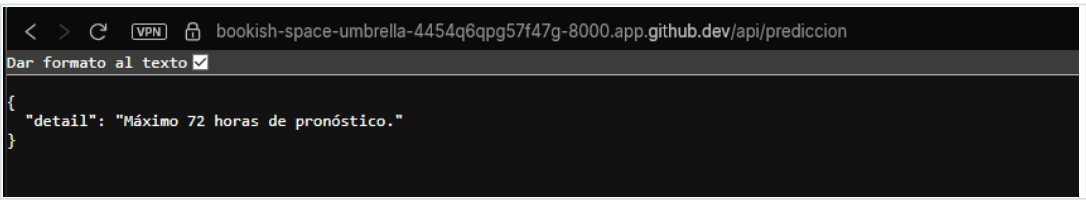



*Figura 20. Respuesta de error de la API (campo detail). El frontend lee este campo y lo muestra en un recuadro de error.*

La app se ejecutó sin Docker, con datos reales de InfluxDB.

```
$ uvicorn app.main:app --host 0.0.0.0 --port 8000
INFO:     Started server process [15767]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     x.x.x.x - "GET / HTTP/1.1" 200 OK
INFO:     x.x.x.x - "GET /api/prediccion?horas=12&historia=48&confianza=0.95 HTTP/1.1" 200 OK
INFO:     x.x.x.x - "GET /api/salud HTTP/1.1" 200 OK
```

*Servidor sin Docker (dirección IP del cliente ocultada).*

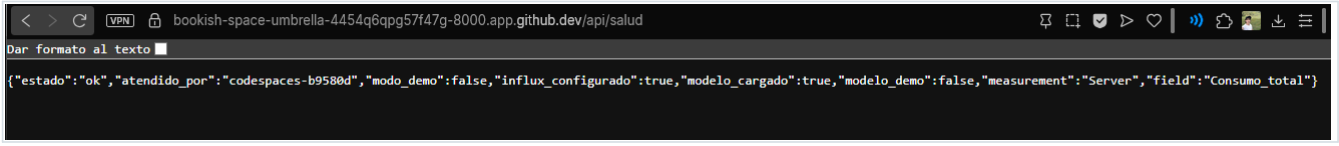



*Figura 21. /api/salud sin Docker: "modelo_demo": false, "modelo_cargado": true, "influx_configurado": true y atendido_por = codespaces-b9580d.*

# 4. Contenedor Docker (Fase 4)

## Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ app/
COPY frontend/ frontend/
COPY models/ models/

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

*requirements.txt se copia antes que el código para que Docker reutilice la capa de pip mientras las dependencias no cambien. El host es 0.0.0.0 para que la app sea accesible desde fuera del contenedor.*

## .dockerignore

```
.env
.env.*
Credenciales*
.git
.gitignore
.github
.devcontainer
.vscode
__pycache__/
*.pyc
*.pyo
.pytest_cache/
.venv/
venv/
notebooks/
scripts/
pruebas.http
README.md
```

*Impide que entren a la imagen las credenciales, el historial de git, los notebooks y las cachés de Python.*

## Construcción y ejecución

```
$ docker build -t pronostico .
[+] Building 41.7s (13/13) FINISHED
 => [internal] load build definition from Dockerfile
 => [internal] load .dockerignore
 => [1/7] FROM docker.io/library/python:3.12-slim
 => [2/7] WORKDIR /app
 => [3/7] COPY requirements.txt .
 => [4/7] RUN pip install --no-cache-dir -r requirements.txt     32.2s
 => [5/7] COPY app/ app/
 => [6/7] COPY frontend/ frontend/
 => [7/7] COPY models/ models/
 => exporting to image
 => => naming to docker.io/library/pronostico
```

*La imagen se construye sin errores (salida abreviada).*

```
$ docker run --rm -p 8000:8000 -e INFLUX_URL -e INFLUX_TOKEN -e INFLUX_ORG -e INFLUX_BUCKET pronostico
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     172.17.0.1:42004 - "GET / HTTP/1.1" 200 OK
INFO:     172.17.0.1:42010 - "GET /api/prediccion?horas=12&historia=48&confianza=0.95 HTTP/1.1" 200 OK
INFO:     172.17.0.1:51748 - "GET /api/salud HTTP/1.1" 200 OK
```

*Las credenciales se entregan al ejecutar (-e), nunca dentro de la imagen.*

## Evidencia del contenedor

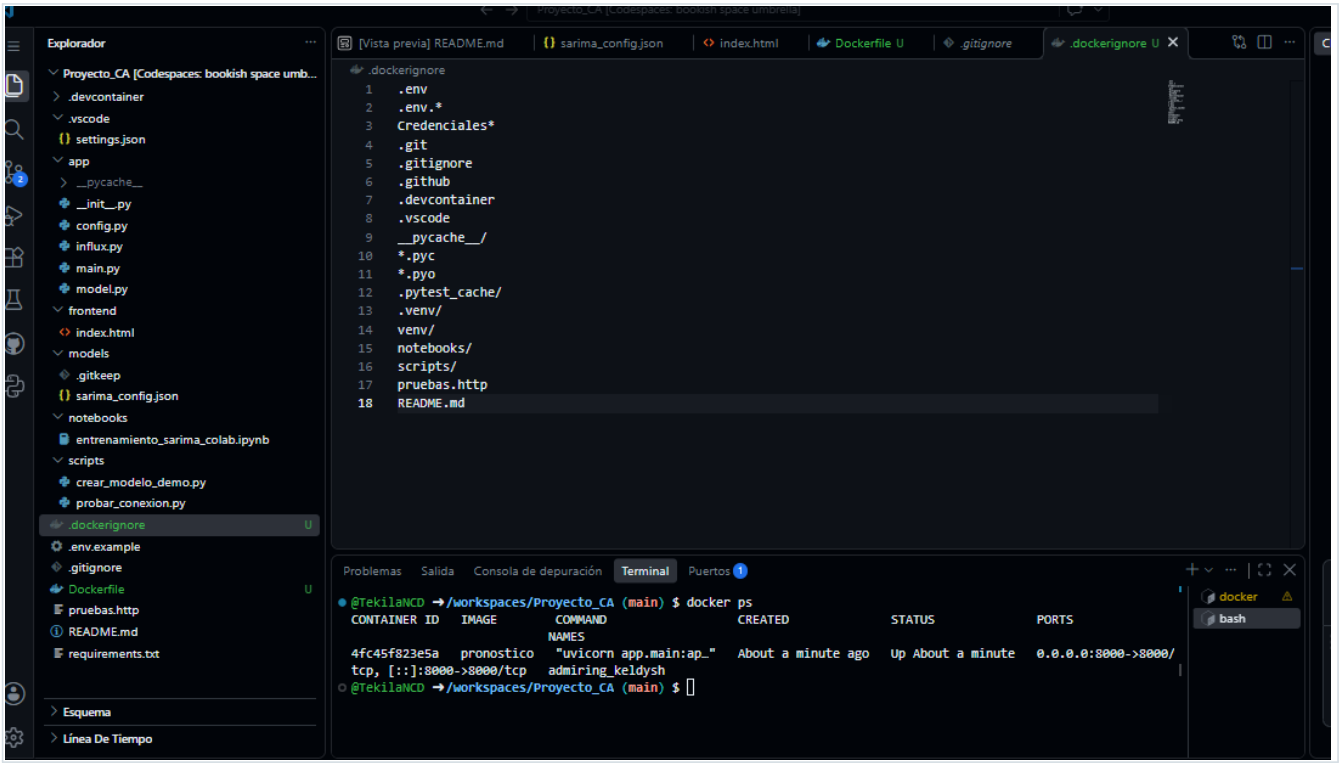



*Figura 22. Dockerfile y .dockerignore en el repositorio, y docker ps con el CONTAINER ID 4fc45f823e5a.*

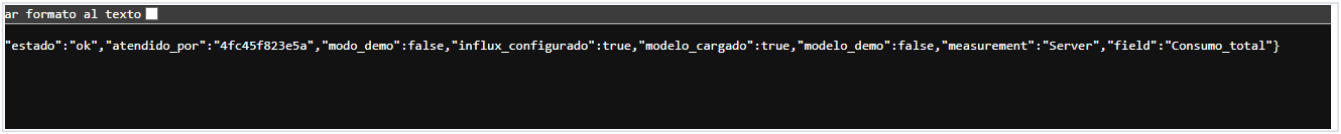



*Figura 23. /api/salud con Docker: atendido_por = 4fc45f823e5a, igual al CONTAINER ID de docker ps y distinto del de la app sin Docker.*

### La imagen no contiene credenciales

```
$ docker run --rm pronostico env | grep -i influx
# (sin salida: la imagen no tiene variables de InfluxDB)

$ docker run --rm pronostico ls -a /app
.
..
app
frontend
models
requirements.txt
```

*Sin variables de InfluxDB y sin archivo .env dentro de la imagen.*

```
$ git add Dockerfile .dockerignore
$ git commit -m "Contenerización del servicio"
[main abaec38] Contenerización del servicio
 2 files changed, 32 insertions(+)
 create mode 100644 .dockerignore
 create mode 100644 Dockerfile
$ git push
   a022172..abaec38  main -> main
```

*Archivos de Docker subidos al repositorio.*

## 5. Verificación de seguridad

Se comprobó que el token no aparece en ningún archivo del repositorio ni en su historial de commits. La comprobación se hizo con la variable de entorno, sin mostrar el token, y se repitió después de subir los archivos de Docker.

```
$ grep -rlF "$INFLUX_TOKEN" . --exclude-dir=.git
$ git log -p | grep -cF "$INFLUX_TOKEN"
0
```

*Sin coincidencias en archivos y 0 coincidencias en el historial.*

# 6. Cumplimiento de la rúbrica

| Criterio (peso) | Evidencia |
|---|---|
| Modelo (15 %) | Modelo propio SARIMA(2,0,2)x(1,1,1,24), notebook ejecutado y verificación con diferencia 0.0 (figuras 10 a 17). |
| Frontend (25 %) | Los cinco requisitos: gráfica con intervalo, hora de Colombia, indicador destacado, control de horas y mensaje de error (figuras 18 a 20). |
| App sin Docker (20 %) | Funciona con datos reales y "modelo_demo": false (figura 21). |
| App con Docker (30 %) | Construye y corre, el frontend funciona, atendido_por = CONTAINER ID y la imagen no tiene credenciales (figuras 22 y 23). |
| Repositorio y video (10 %) | Repositorio con modelo, notebook ejecutado, frontend, Dockerfile y .dockerignore. El video se entrega aparte. |